

Documentation Outline (for Word Doc / GitHub README)

Table of Contents

1. Project Overview
2. Business Objectives
3. Understanding the Domain
4. Data Collection (Contacts dataset)
5. Data Cleaning
6. Feature Engineering
7. Exploratory Data Analysis (EDA)
8. Visualization
9. Interpretation
10. Code Implementation
11. How to Run the Project
12. Future Improvements

1. Project Overview

This project is a **Contact Book Application** built in Python. It simulates a dataset of contacts and allows CRUD operations (Create, Read, Update, Delete). It demonstrates fundamental data engineering steps: collection, cleaning, feature engineering, and analysis.

2. Business Objectives

- Manage personal/professional contacts efficiently.
- Provide a structured dataset for analysis.
- Demonstrate Python programming and data handling skills.

3. Understanding the Domain

Contacts are a form of structured data: **Name, Phone, Email, Address**. Managing them requires consistency, validation, and easy retrieval.

4. Data Collection

Instead of an external dataset, we **collect data interactively** via user input. Each contact is stored in a Python dictionary.

5. Data Cleaning

- Prevent duplicate entries.
- Validate missing fields.
- Allow editing with defaults if fields are left blank.

6. Feature Engineering

- Add derived features (e.g., count of contacts, grouping by domain of email).
- Potential to extend with tags, categories, or relationship type.

7. EDA

- Summarize number of contacts.
- Explore distribution of email domains (e.g., Gmail vs Yahoo).
- Identify missing or incomplete records.

8. Visualization

- Could be extended with matplotlib/seaborn to visualize email domain distribution.

9. Interpretation

- A clean, structured contact dataset improves communication efficiency.
- Demonstrates how CRUD operations mirror real-world data pipelines.

Full Python Code (with comments for VS Code)

python

Contact Book Application

Author: Victor

Description: A simple Python project to manage contacts (CRUD operations).

This project simulates a dataset and demonstrates data engineering workflow.

Function: Display Menu

```

# -----
# Imports
# -----

import csv

# -----
# Helper Functions: Save/Load Dataset
# -----

def save_to_csv(contact_book, filename="contacts.csv"):
    with open(filename, mode="w", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(["Name", "Phone", "Email", "Address"])
        for name, details in contact_book.items():
            writer.writerow([name, details["phone"], details["email"], details["address"]])
        print("Contacts saved to contacts.csv")

def load_from_csv(filename="contacts.csv"):
    contact_book = {}
    try:
        with open(filename, mode="r") as file:
            reader = csv.DictReader(file)
            for row in reader:
                contact_book[row["Name"]] = {
                    "phone": row["Phone"],
                    "email": row["Email"],
                    "address": row["Address"]
                }
    
```

```

        print("Contacts loaded from contacts.csv")
    except FileNotFoundError:
        print("No dataset found, starting fresh.")
    return contact_book

# -----
# Core Contact Book Functions
# -----

def display_menu():
    print("\nContact Book Menu:")
    print("1. Add Contact")
    print("2. View Contact")
    print("3. Edit Contact")
    print("4. Delete Contact")
    print("5. List All Contacts")
    print("6. Exit")

def add_contact(contact_book):
    name = input("Enter Name: ")
    if name in contact_book:
        print("Contact already exists!")
        return
    phone = input("Enter Phone: ")
    email = input("Enter Email: ")
    address = input("Enter Address: ")
    contact_book[name] = {"phone": phone, "email": email, "address": address}
    print("Contact added successfully!")

```

```
def view_contact(contact_book):  
    name = input("Enter Name to View: ")  
    if name in contact_book:  
        contact = contact_book[name]  
        print(f"\nName: {name}")  
        print(f"Phone: {contact['phone']}")  
        print(f"Email: {contact['email']}")  
        print(f"Address: {contact['address']}")  
    else:  
        print("Contact not found!")
```

```
def edit_contact(contact_book):  
    name = input("Enter Name to Edit: ")  
    if name in contact_book:  
        phone = input("Enter New Phone (leave blank to keep current): ")  
        email = input("Enter New Email (leave blank to keep current): ")  
        address = input("Enter New Address (leave blank to keep current): ")  
  
        if phone == "":  
            phone = contact_book[name]["phone"]  
        if email == "":  
            email = contact_book[name]["email"]  
        if address == "":  
            address = contact_book[name]["address"]  
  
        contact_book[name] = {"phone": phone, "email": email, "address": address}
```

```

        print("Contact updated successfully!")
    else:
        print("Contact not found!")

def delete_contact(contact_book):
    name = input("Enter Name to Delete: ")
    if name in contact_book:
        del contact_book[name]
        print("Contact deleted successfully!")
    else:
        print("Contact not found!")

def list_all_contacts(contact_book):
    if not contact_book:
        print("No contacts available.")
    else:
        print("\nAll Contacts:")
        for name, details in contact_book.items():
            print(f"Name: {name}")
            print(f"Phone: {details['phone']}")
            print(f"Email: {details['email']}")
            print(f"Address: {details['address']}")
            print()

# -----
# Main Program Loop
# -----

```

```
contact_book = load_from_csv() # Load dataset at start

while True:
    display_menu()
    choice = input("Enter your choice (1-6): ")
    if choice == "1":
        add_contact(contact_book)
    elif choice == "2":
        view_contact(contact_book)
    elif choice == "3":
        edit_contact(contact_book)
    elif choice == "4":
        delete_contact(contact_book)
    elif choice == "5":
        list_all_contacts(contact_book)
    elif choice == "6":
        save_to_csv(contact_book) # Save dataset before exit
        print("Exiting Contact Book. Goodbye!")
        break
    else:
        print("Invalid choice. Please try again.")
```